CAPSEL / INCA

Master Reference Document

Continually-growing transformer language model with saturation-driven architectural expansion

Nishant Kumar · PhD Programme · May 2026

Replaces: CAPSEL_research_memorandum_2.docx, CAPSEL_research_memorandum_FINAL.docx, CAPSEL_research_memorandum_FINAL_v2.docx

Core Thesis Statement

A continually-learning language model should allocate architectural capacity as a **principled, multi-signal decision** — not a schedule — growing when a consensus of calibrated signals declares its current capacity exhausted, sharing knowledge across frozen experts through **function-preserving lateral adapters**, replaying memories under a **biologically-grounded schedule**, routing over the resulting expert pool with **uncertainty-calibrated load-balanced gates**, and bounding its own size through **principled merging of representationally-redundant experts**.

> *"Measure, do not threshold. Allocate, do not schedule. Share laterally, not only sequentially. Merge, do not hoard."*

1. The Problem

1.1 Why standard continual learning fails for language models

Large language models trained sequentially on evolving data suffer from **catastrophic forgetting**: gradient updates for a new period overwrite weights that encoded knowledge from prior periods. The stability-plasticity dilemma is especially acute for LLMs because (a) parameter counts are enormous relative to per-period data, (b) temporal knowledge conflicts are common (facts change over time), and (c) standard regularisation methods (EWC, L2P) add computational overhead that scales badly with model size.

1.2 What is missing from the 2024–2026 literature

- **No principled when-to-grow signal.** Every dynamic-expansion method (LLaMA-Pro, LiGO, D-MoLE) uses a fixed schedule or manually-chosen checkpoint — not a model-internal saturation signal.
- **No cross-block knowledge transfer.** Frozen blocks in growing architectures are treated as opaque prefixes. No published method gives the trainable block a parameter-light direct path to every prior frozen representation.
- **No CLS-grounded replay for LLMs.** Existing replay buffers use uniform or hardest-only selection. Neither matches biological hippocampal scheduling.
- **No bounded-size continual growing system.** Growing architectures eventually become prohibitively large. No published method merges redundant blocks with provable post-merge routing error bounds.

2. INCA — The Core Idea

2.1 What INCA is

INCA (**Incremental Neural Chain Architecture**) grows a transformer encoder vertically — one block at a time — driven entirely by model-internal signals. When the current trainable block saturates (the model has learned everything the current data period can teach it), that block is frozen and a warm-started copy becomes the new trainable block. A **CrossAttentionSelector** aggregates all block outputs into a single encoder representation passed to the T5 decoder.

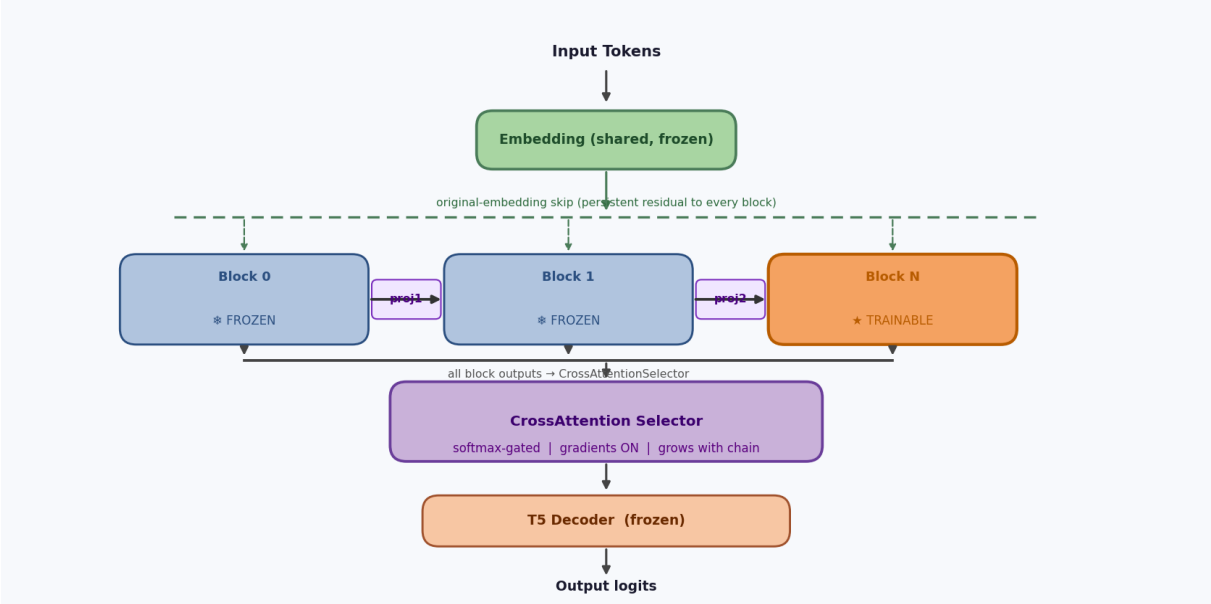
2.2 Why this design

Design choice	Why
Freeze-and-grow (not fine-tune-in-place)	Frozen blocks act as neocortical long-term memory (CLS theory). New trainable block starts from warm-start copy — fast initial convergence.
Sequential chain + embedding skip	Each block builds on prior knowledge via chain. Original-embedding skip prevents representation drift through deep frozen chains.
Saturation-driven growth (not scheduled)	Fixed schedules waste parameters on easy periods, underfit hard ones. Multi-signal consensus correctly times growth.
Lateral adapters (Phase 2)	Frozen blocks cannot be queried directly by the new block through the chain only. Rank-r adapters provide a direct, parameter-light path from any prior frozen block.
Study-schedule replay	Hippocampus preferentially replays well-encoded episodes (biological CLS). Two-phase schedule: uniform early, then hard-dominated.
CKA monitoring	Representation drift measured against a fixed 200-item reference set. Feeds into the saturation consensus AND into router staleness prior.

3. Architecture

3.1 Current architecture (a) — sequential chain + embedding skip

Every block receives the output of the previous block (through an identity-initialised projection **proj_i**) PLUS the original token embeddings as a direct skip. The key constraint: there is **no direct Block i → Block j (j > i+1)** connection in architecture (a). All blocks feed their final hidden states into the CrossAttentionSelector independently.



3.2 Forward pass (pseudocode)

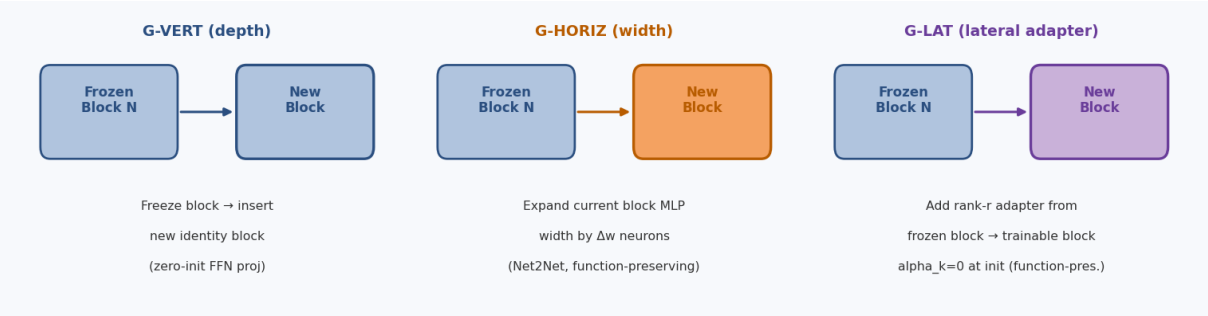
```
embedding_hidden = embed_tokens(input_ids) # shared, frozen
chain_hidden = embedding_hidden # Block 0 input

for i, block in enumerate(self.blocks):
if i > 0:
# inter-block projection + embedding skip
proj = inter_block_projs[i-1] # identity-init at grow time
chain_hidden = proj(chain_hidden) + embedding_hidden
h = block(chain_hidden, attention_mask) # T5Block forward
h = final_layer_norm(h)
chain_hidden = h # pass to next block
block_outputs.append(h)

return selector(block_outputs, attention_mask) # CrossAttentionSelector
```

3.3 Growth primitives

Three primitives are available when the saturation detector fires. The SPRT margin and frozen-block CKA drift determine which is selected.



Primitive	Trigger condition	Effect	Function-preserving?
G-VERT (depth)	Large margin + low frozen CKA drift	Freeze block, insert new identity block (zero-init FFN proj)	Yes — exactly
G-HORIZ (width)	Large margin + high CKA drift	Expand current MLP by Δw neurons via Net2Net	Yes — by construction
G-LAT (lateral)	Small-but-persistent margin	Add rank-r adapter from frozen block, alpha_k=0 at init	Yes — at init
G-EXPERT (Paper B)	Block-full in expert-pool mode	Add new block B_{n+1} warm-started from B_n	Yes — warm-start

3.4 Architecture (c) — lateral adapters (Phase 2 target)

In architecture (c), the trainable block additionally receives low-rank adapter outputs from **every** earlier frozen block. Each adapter A_k is a rank-r linear mapping gated by a learned scalar α_k that starts at zero (function-preserving). This gives the current block direct access to any prior frozen representation, resolving the chain-depth distortion problem.

```
# Architecture (c) — lateral adapter input to current block Fc:
h_fc_input = chain_hidden # standard chain input
for k, (frozen_block_out, adapter) in enumerate(zip(frozen_outputs, adapters)):
h_fc_input = h_fc_input + alpha_k * adapter(frozen_block_out)
h_fc = trainable_block(h_fc_input, attention_mask)

# Each alpha_k is a learned scalar, initialised at 0.0
# adapter = nn.Linear(d_model, d_model, bias=False) with rank-r factorisation
```

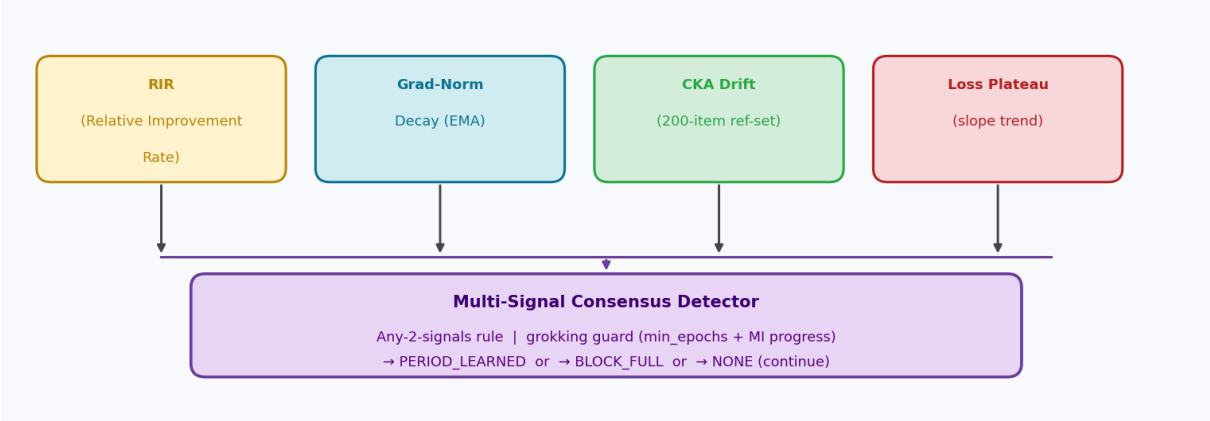
4. Saturation Detection

4.1 Why thresholds are not defensible

The original detector used fixed accuracy thresholds (80% for PERIOD_LEARNED, 60% for BLOCK_FULL). On RealtimeQA, a well-trained FLAN-T5-base reaches only **~30–35% token F1** — permanently below the 60% threshold, so BlockFull never fires. The 50-probe confidence band is **±10%**, so a model at 74% true accuracy triggers PERIOD_LEARNED by chance ~35% of the time within ten evaluations. The signal is statistically indefensible.

4.2 Multi-signal consensus (Paper A)

Four signals are combined under an any-2-signals rule:



Signal	Formula	Interpretation
RIR (Relative Improvement Rate)	r_t = (score_t – score_0) / max(score_0, 0.25)	Dataset-agnostic improvement since period start. chance=0.25 for 4-cl
Grad-Norm decay (EMA)	g_t = EMA(‖∇θ‖■) / g_initial	Training signal strength. Decays as block saturates.
CKA drift	r_t = CKA(H_t, H_ref) on 200-item reference set	Representation stability. Near 1.0 = block not learning. Drops when dri
Loss plateau	Slope of EMA-smoothed training loss	Classic plateau signal. Already implemented.

Event	Fires when
PERIOD_LEARNED	RIR > 30% improvement AND loss plateau holds
BLOCK_FULL	RIR negligible AND loss plateau AND (grad-norm decayed OR CKA saturated)
NONE	Continue training

4.3 Grokking guard

The controller may not declare H1 (block saturated) until: (1) epoch count exceeds **min_epochs** (default 10), AND (2) information-theoretic progress measure MI_t – MI_0 ≥ δ_I (~0.05 nats), where MI_t = I(H_t(X); Y) estimated via MINE. This prevents firing the saturation signal during the grokking memorisation phase.

4.4 SPRT extension (optional — Paper B)

Wald's Sequential Probability Ratio Test replaces 7 hand-tuned hyperparameters with 2 calibrated error rates α=0.05, β=0.10, giving thresholds A≈2.89, B≈–2.94:

```
Λ_t = Σ log[ p1(s_τ) / p0(s_τ) ] where s_τ = (g,r,f,i) ∈ R■
= (μ1–μ0)■ Σ■¹ · Σ_τ ( s_τ – (μ0+μ1)/2 )

Declare H1 (block full) if Λ_t ≥ A = log((1–β)/α) ≈ 2.89
Declare H0 (capacity remains) if Λ_t ≤ B = log(β/(1–α)) ≈ –2.94

Expected detection time: E[T|H1] ≤ A / D(p1||p0)
where D = (1/2)(μ1–μ0)■ Σ■¹ (μ1–μ0) [Mahalanobis divergence]
```

5. Replay — the CLS Mechanism

5.1 Core principle — buffer is per-block, not global

The replay buffer exists for **one purpose only**: prevent the current trainable block from overwriting an earlier period while it is still being trained on a newer one — i.e. *within-block forgetting*.

A frozen block's weights already encode everything it learned. Replaying raw examples from a frozen period to a new, different trainable block is meaningless — the new block is not trying to re-learn the frozen block's knowledge, it builds on top of it. Therefore the buffer is cleared completely at every freeze event. The frozen block IS the replay.

Event	Buffer action	Reason
Period k learned inside Block i	Add period-k items to buffer	Block i is still trainable; future periods may land here too
Block i still trainable, new Period k+1 starts	Replay period-k items into training batches	Prevent Block i forgetting period k while learning period k+1
Block i saturates → FREEZE	Clear entire buffer	Frozen weights ARE the memory. Raw examples no longer needed.
Block i+1 (new trainable block) starts	Buffer starts empty	Fresh block, fresh slate — no cross-block replay

5.2 Lifecycle example (3 blocks, 6 periods)

```
Block 0 trains on Period 1 → learned → add P1 to buffer
Block 0 trains on Period 2 → replay P1 items mixed into batches
Block 0 saturates → FREEZE → buffer cleared (Block 0 weights = P1+P2 memory)

Block 1 starts, buffer empty
Block 1 trains on Period 3 → learned → add P3 to buffer
Block 1 trains on Period 4 → replay P3 items mixed into batches
Block 1 saturates → FREEZE → buffer cleared (Block 1 weights = P3+P4 memory)

Block 2 starts, buffer empty
Block 2 trains on Period 5 → learned → add P5 to buffer
Block 2 trains on Period 6 → replay P5 items mixed into batches
...
```

Buffer size is bounded by periods_per_block × max_items_per_period and never grows beyond one block's worth of periods. Contrast with naive global replay buffers that grow indefinitely with every new period.

5.3 CLS mapping

INCA component	Biological analogue	CLS experiment
Frozen block weights	Neocortex — distributed semantic store	E-CLS2 (linear probing)
Replay buffer (within-block only)	Hippocampus — short-term episodic memory	E-CLS1, E-CLS3
Buffer clear at freeze	Hippocampal pruning after consolidation	E-CLS5
Freeze-and-grow event	Systems consolidation	E-CLS4, E-CLS5
Sequential block merging (Paper B)	Semantic integration across neocortex	NOT CLS — distinct

5.4 Study-schedule sampling (two-phase)

Within a block's training the buffer samples using a study schedule. The buffer is small (only within-block periods), so every sample matters.

Phase	Epochs	Sampling rule	Rationale
Initial-pass (A)	1 – N_revise (default 3)	Uniform random from buffer	Revise everything broadly before deciding what needs wor
Refinement (B)	N_revise onward	70% hard + 20% easy + 10% mid	Hard: fix weakening memories. Easy: prevent silent fade. M

Why keep easy items (20%)? If the buffer only replays hard items, easy items quietly degrade below the detection threshold. By the time they appear hard, they are already forgotten. Easy-item maintenance prevents silent forgetting. The biological hippocampus replays well-encoded episodes precisely to consolidate them — not just the confusing ones.

5.5 T1.3 — Period-transition drift check

After a new period starts inside an existing block, every K_eval steps the training loop checks whether the current period's training is actively damaging the items already in the buffer. If replay accuracy drops more than 10% relative to where it was at period start, BLOCK_FULL fires early — the block is saturated and cannot absorb another period without damaging the previous one.

```
# At period-advance (only if buffer already has items from this block):
had_replay_before = bool(replay_buf.all_items())
replay_acc_before = eval_accuracy(model, replay_buf.all_items())

# During training, every K_eval steps:
if had_replay_before:
current_acc = eval_accuracy(model, replay_buf.all_items())
if current_acc < (1 - period_drift_tol) * replay_acc_before:
trigger BLOCK_FULL # forward interference — freeze now
# period_drift_tol default = 0.10 (10% relative drop allowed)
```

6. Routing / Selector

The selector aggregates all block outputs — frozen and trainable — into a single (B, S, D) tensor for the decoder. Four variants are implemented from Phase 1 and ablated in E-ROUTE. See the Selector Architecture document for full diagrams.

6.1 Selector variants (E-ROUTE ablation)

YAML key	Name	Granularity	Core idea	Default
embedding_query	S-QKV	Token-level	Q = frozen embeddings (fixed). K, V from each block separately. Q attends to each block	✓ Yes
uclbr	UCLBR	Token-level	Extends S-QKV with: Read-ME pre-gate (relevance filter), DeepSeek load-balance bias	
cross_attention	S-FULL	Sequence-level	MLP gate on mean-pooled block output → scalar per block. Softmax. Same weight for	Ablation
weighted_sum	S-WS	Sequence-level	One learnable scalar per block. Input-independent. Blind control — no content awareness	Control

6.2 Why S-QKV is the default

- **Fixed Q = original meaning anchored.** The frozen embedding asks: "given what this token originally meant, which block serves it best?" Q never drifts regardless of training length.
- **Per-block separate attention.** Q attends to Block 0, Block 1, Block 2 independently. Each block gets its own Attention(Q, K_i, V_i) → out_i. No token mixing across blocks in a single attention step.
- **Token-level routing.** Token at position 3 can favour Block 0 while token at position 15 favours Block 2. S-FULL cannot do this — it uses one weight for all positions.
- **Identity-initialised projections.** W_k and W_v start as identity matrices, making the selector function-preserving at grow time.

6.3 UCLBR — three components

- **Read-ME pre-gate:** r_i = sigmoid(MLP(mean_pool(block_i))). Relevance score ∈ (0,1) per block. Routing logits multiplied by r_i before softmax — irrelevant frozen blocks suppressed before full attention.
- **DeepSeek load-balance bias (no aux loss):** buffer lb_bias_i updated online: lb_bias_i ← lb_bias_i + η·(1/n − f_i) where f_i = mean routing fraction to block i over the batch. Under-used blocks get higher bias. No term added to training loss.
- **Uncertainty-calibrated confidence:** H■ = normalised entropy of routing weights. c = learned(H■). w_final = c·w_router + (1−c)·uniform. When router is uncertain, output falls back toward equal weighting.

7. Pruning Before Freeze

At freeze time: compute diagonal Fisher over the replay buffer on the current block, then structurally prune the bottom p% of MLP neurons and attention heads by Fisher score. Structured pruning (not unstructured) because only structured pruning gives wall-clock speedup. Ablation sweeps p ∈ {0, 10, 20, 30}%; expected sweet spot 10–20%.

```
def freeze_and_grow(self):
F = diagonal_fisher(current_block, replay_batches) # true Fisher (not empirical)
head_scores = F.attn_out_proj.sum(dim=col_per_head)
neuron_scores = F.mlp_c_fc_output.sum(dim=0)
```

```
prune_heads (block, head_scores, keep_frac=1.0 - p_heads) # p_heads ∈ {0,..,0.3}
prune_neurons (block, neuron_scores, keep_frac=1.0 - p_neurons) # p_neurons ∈ {0,..,0.3}
for q in block.parameters(): q.requires_grad = False
inter_block_projs.append(identity_projection(d_model)) # new proj, frozen
```

Soen et al (2025): use labelled targets (not model's own predictions); batch-size-normalise; unit-test against known closed form.

8. Paper A — Ablations

ID	Name	What it tests	Pass criterion
E-SAT	Saturation detector comparison	threshold-INCA vs RIR-only vs multi-signal vs SPRT	Multi-signal reduces FPR+FNR below threshold-INCA on ≥3/4 benchmarks; ≥30% fewer spurious growth triggers
E-SAT-AGNOSTIC	Dataset-agnosticism	Same config on RealtimeQA (~30–35% F1), StreamingQA (~40–45%), TemporalWiki (~50%)	Both detectors fire on ≥2/3 datasets with no hyperparameter changes
E-SIG	Signal contribution	Ablate each of 4 signals independently + in pairs	g and r strongest; f adds robustness; i handles dataset-agnosticism
E-GROW	Growth primitive	G-VERT only / G-HORIZ only / G-LAT only / SPRT-margin-driven mix	Mixed strategy is Pareto-dominant on params vs BWT
E-SCOPE	Lateral-adapter scope	S-FULL / S-ATTN / S-MLP / S-NONE; rank $r \in \{4,8,16\}$	Identify which frozen-block features carry most transferable knowledge
E-CLS1	Replay necessity	Replay off vs buffer sizes {100,500,2000,all}	Replay necessary; diminishing returns visible
E-CLS2	Linear probing frozen blocks	Linear probe accuracy on each frozen block	(a) monotone decrease block 0→N; (b) above-chance throughout
E-CLS3	Replay strategy	Uniform / hardest-only / easiest-only / study-schedule	Study-schedule best or within 1 pt; CLS-motivated
E-CLS4	Consolidation dynamics	Per-epoch: train loss, val loss, replay loss, CKA	Replay loss drops before val loss rises
E-CLS5	Forgetting curves	Power-law fit on BWT per frozen block over time	Power-law decay confirmed; slope faster for naive fine-tuning
E-PRUNE	Pruning fraction sweep	$p \in \{0,10,20,30\}$%; Fisher vs magnitude	Fisher outperforms magnitude; sweet spot 10–20%
E-ROUTE	Routing ablation	R-CA / R-WS / R-GT / R-HB / R-UB; primary metric BWT+ACC	R-UB expected winner; routing calibration diagrams included
E-TRANS	Period-transition safety	With vs without replay drift check (T1.3)	Drift check reduces BWT drop after same-block period advance
E-GROK	Grokking stress test	Continual modular arithmetic; CAPSEL with/without grokking guard	Guard prevents block-full during memorisation phase
E-SCALE	Scale validation	FLAN-T5-large (780M, Track A); Pythia-160M + GPT-2-Medium (Track B)	Results hold at larger scale

9. Paper B — Sequential Block Merging

Every K grow events (recommended K=3), check all frozen-block pairs for redundancy via CKA. If CKA(B_i, B_j) > τ_merge (default 0.9), merge using three steps:

- **1. Geometric merge:** TIES-style trim-and-sign-elect of delta vectors relative to shared warm-start ancestor. Fallback: SLERP weighted by routing frequency.
- **2. Distillation refinement:** fine-tune merged block θ_{ij} to minimise KL divergence against average output of B_i and B_j on replay pool.
- **3. Router recalibration:** freeze merged block + all others; train only router on short pass over held-out stream data.

9.1 Post-merge router error bound (theorem)

Assumptions: router R is L-Lipschitz in feature embedding; merged block B_m satisfies ||B_m(x)−(B_i(x)+B_j(x))/2|| ≤ ε_m; CKA(B_i,B_j) ≥ 1−ε_c. **Result:** router output distribution after recalibration differs from pre-merge by at most a constant depending on (ε_m + ε_c)·L.

9.2 E-MERGE acceptance criterion

Train CAPSEL to 12 periods with/without merging. Merging passes if: model size stays within **factor of 2** of 6-period baseline AND BWT loss ≤ **2 percentage points**.

10. Baselines

10.1 Phase 0 baselines (B1–B8) — all code complete

ID	Name	Key parameter	What it isolates
B1	Naive fine-tuning	~250M (FLAN-T5-base)	Forgetting floor (lower bound)
B2	Replay-only	~250M	Replay without architecture change
B3	EWC	~250M	Regularisation vs replay
B4	L2P (Learning to Prompt)	Soft prompts only ~0.1% params	Parameter-efficient adaptation with frozen backbone
B5	LoRA-MoE	LoRA experts ~1% params/period	Modular routing without block growth
B6	LLaMA-Pro (vertical)	One identity block/period	Scheduled vertical growth — the paper to beat
B7	Progressive Neural Networks	Full column/period	Lateral connections baseline; lossless upper bound on BWT at linear param cos
B8	BlockStack variant	One block/growth event	Paper A internal reference; frozen stack + trainable current + pluggable selector

10.2 Modern baselines (secondary — for 2026 review)

- SEEKR (He et al, EMNLP 2024) — current SOTA on TRACE benchmark
- Online-LoRA (Wei et al, WACV 2025) — task-free loss-dynamics + LoRA
- InfLoRA (Liang et al, CVPR 2024) — interference-free LoRA subspace
- D-MoLE (Ge et al, ICML 2025) — dynamic LoRA rank allocation per layer
- MIGU (Du et al, EMNLP Findings 2024) — task-free magnitude gating
- DER++ (Buzzega et al, NeurIPS 2020) — logit-distillation replay
- LLaMA-MoE (Zhu et al, EMNLP 2024) — converted LLaMA MoE baseline
- CL-MoE (Huai et al, CVPR 2025) — dual-router for CL-VQA

Bounds: naive sequential fine-tuning (lower) · joint training on all periods (upper) · frozen pretrained FLAN-T5-base (forward-transfer reference).

11. Benchmarks and Datasets

Dataset	Source	Role	Notes
TiC-LM	Li et al, ACL 2025 Oral (arXiv 2504.02107)	PRIMARY (Paper A)	2.9T tokens; 114 monthly Common Crawl slices; run 6–12 consecutiv
TRACE	Wang et al, 2023 (arXiv 2310.06762)	Secondary	8 tasks: domains, code, math, multilingual; instruction-following degra
TemporalWiki	Dhingra et al, EMNLP 2022	Secondary	Year-half periods; UpdatedLAMA-style cloze; gradual drift
CKL	Jang et al, ICLR 2022 (arXiv 2110.03215)	Secondary	InvariantLAMA / UpdatedLAMA / NewLAMA knowledge probes
RealtimeQA	Kasai et al, NeurIPS 2022	Phase 0 Track A	Year-week periods; MC + open-answer; F1 ceiling ~30–35% with FLA
StreamingQA	Liska et al, 2022	Phase 0 secondary	CC-News 2007–2021; monthly periods; open-answer; F1 ceiling ~40–
CC-News	HF: vblagoje/cc_news	Phase 0 current	Year-month periods; currently used for smoke runs

Dataset	Source	Role	Notes
Modular arithmetic	Constructed from Nanda et al 2023	E-GROK only	Grokking stress test — 5 tasks with increasing modulus

12. Implementation Status

12.1 Phase 0 — Complete

File	Status	Notes
Phase0/baselines/b1_finetune.py	✓ Running	Seq2seq + causal; logging fixed
Phase0/baselines/b2–b7	✓ Code complete	Awaiting full baseline run
Phase0/common/runner.py	✓ Fixed	Epoch logging; folder wipe on restart
Phase0/common/harness.py	✓ Fixed	RunLogger wipes old run on init
Phase0/common/metrics.py	✓	CAPSEL XIII: BWT/ACC/FWT/RIR/CKA/Fisher
Phase0/data/download_cc_news.py	✓	Stream-to-disk; checkpoint/resume
Phase0/data/preprocess_cc_news.py	✓	Quality filters; 3 probe types

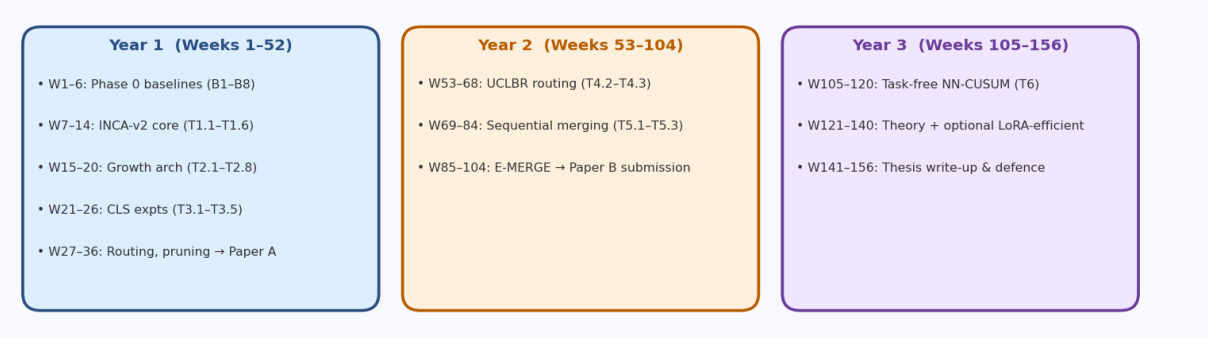
12.2 Phase 1 — Complete (smoke run passing)

File	Status	Notes
Phase1/src/inca_config.py	✓	52 hyperparams; INCAConfig dataclass
Phase1/src/inca_plateau.py	✓ 17 tests pass	RIR+GradNorm+CKA+loss consensus
Phase1/src/inca_replay.py	✓ 13 tests pass	Study-schedule; phase A/B sampling
Phase1/src/inca_cka.py	✓	Unbiased HSIC; 200-item ref-set
Phase1/src/inca_selectors.py	✓	CrossAttn/Weighted/Gated selectors
Phase1/src/inca_layer_manager.py	✓ patched	Arch (a): sequential chain + emb skip; inter_block_projs
Phase1/src/train_inca_v3.py	✓ smoke pass	T1.1–T1.3; gnorm/T1.3 bugs fixed
Phase1/scripts/visualize_inca.py	✓	Block diagram from .pt checkpoint

12.3 Phase 2–5 — Pending

Phase	Key deliverables	Status
Phase 2 — Growth architecture	inca_lateral.py; inca_width.py; inca_growth_chooser.py; E-GROW; E-SCOPE	Not started
Phase 3 — CLS experiments	probe_frozen_blocks.py; E-CLS1–E-CLS5 results	Not started
Phase 4 — Routing + Paper A	inca_uclbr.py; E-ROUTE; E-SAT; E-SCALE; Paper A draft	Not started
Phase 5 — Paper B	Block merging; router recalibration; E-MERGE; Paper B draft	Year 2

13. Three-Year Roadmap



Milestone	Target	Venue
Phase 0 baselines complete	Week 6	Internal
Phase 1 INCA-v2 smoke run	Week 14	✓ Done
Phase 2 lateral adapters + growth	Week 20	Internal
Phase 3 CLS experiments	Week 26	Internal
Paper A submitted	End of Year 1	NeurIPS / ICLR / ICML main track
Paper A accepted / R&R	Year 2	
Sequential merging + E-MERGE	Month 21	Internal
Paper B submitted	Month 24	NeurIPS Efficient ML / ICML main track
Task-free NN-CUSUM	Month 27	
Thesis defended	Month 36	

Compute budget: ~6 GPU-months Year 1 · ~10 Year 2 · ~6 Year 3 = ~22 GPU-months total. Compressed budget (TiC-LM 6 months instead of 12): ~12 GPU-months total.

14. Key Numbers Reference

Parameter	Value	Where used
CKA reference set size	200 items	inca_cka.py; all CKA computations
CKA saturation threshold	0.95	BLOCK_FULL via CKA signal
RIR improvement target	30%	PERIOD_LEARNED condition
period_drift_tol	0.10 (10% relative drop)	T1.3 replay drift check — fires BLOCK_FULL early
Replay scope	Per-block only — cleared at freeze	Buffer size bounded; frozen weights = memory
Study-schedule N_revise	3 epochs	Phase A (uniform) → Phase B (study schedule) transition
Study-schedule p_hard	0.70	Phase B: highest-loss items (weakest memories)
Study-schedule p_easy	0.20	Phase B: lowest-loss items (prevent silent fade)
Study-schedule p_mid	0.10	Phase B: random mid-loss coverage
SPRT α	0.05	E[FP] = 5%
SPRT β	0.10	E[FN] = 10%
SPRT A threshold	≈ 2.89	$\log((1-\beta)/\alpha)$
SPRT B threshold	≈ -2.94	$\log(\beta/(1-\alpha))$
Grokking guard δ_l	~0.05 nats	MI progress floor
Fisher pruning sweet spot	10–20%	E-PRUNE ablation
Merge CKA threshold τ_merge	0.90	Paper B merge trigger
Merge cadence K	3 grow events	Paper B
UCLBR load-balance η_b	1e-3	Router bias update
Lateral adapter rank r	{4, 8, 16} (ablated)	E-SCOPE
Lateral adapter α_k init	0.0 (function-preserving)	inca_lateral.py
Primary model (Track A)	google/flan-t5-base (250M)	All Phase 0 + Paper A
Scale model (Track A)	google/flan-t5-large (780M)	E-SCALE
Primary model (Track B)	Pythia-160M	Phase 0 Track B
E-SCALE model (Track B)	GPT-2 Medium (345M)	E-SCALE
Number of seeds	3	All headline experiments

15. Open Bugs and Audit Findings

ID	Finding	Severity	Fix
A1	T1.3 false-fires in Period 1 (had_replay_before flag missing)	HIGH — fixed	had_replay_before guard added to train_inca_v3.py
A2	gnorm measured after zero_grad (always 0.0)	HIGH — fixed	last_grad_n captured before zero_grad
A3	T5Block.forward() called with head_mask (removed in new transformers)	HIGH — fixed	head_mask removed from INCABlock.forward()
A4	T5Attention needs cache_position in new transformers	HIGH — fixed	cache_position=torch.arange(seq_len) passed to each layer
A5	inca_layer_manager forward was fully parallel (not sequential)	HIGH — fixed	Architecture (a) implemented: chain + embedding skip + inter
A6	RunLogger accumulated data from previous runs	MEDIUM — fixed	shutil.rmtree(out_dir) added to RunLogger.__init__
A7	No epoch-level logging in training.log	MEDIUM — fixed	text_logger callback threaded through runner→baseline→trai
A8	Selector normalisation-by-count (out / n dilution)	MEDIUM — open	Replace with softmax over gate logits (Phase 2 task)
A9	HebbianSelector not wired to CLI	LOW — open	Wire --selector hebbian in Phase 4 (T4.1)
A10	INCAConfig missing out_dir field (YAML parse error)	LOW — fixed	Added out_dir: str = 'Phase1/results' to INCAConfig

16. Mandatory Reading List

#	Paper	arXiv	Why mandatory
1	LLaMA-Pro (Wu et al, ACL 2024)	2401.02415	Direct prior art — paper INCA must beat
2	Online-LoRA (Wei et al, WACV 2025)	2411.05663	Closest methodological sibling
3	Spurious Forgetting (Zheng et al, ICLR 2025)	—	Justifies freeze-below-grow-above topology
4	Grokking (Nanda et al, ICLR 2023)	2301.05217	Basis for grokking guard
5	TiC-LM (Li et al, ACL 2025 Oral)	2504.02107	Primary benchmark
6	TRACE (Wang et al, 2023)	2310.06762	General CL-LLM capability benchmark
7	SEEKR (He et al, EMNLP 2024)	2411.06171	2024 SOTA replay-efficient method
8	LoRAMoE (Dou et al, ACL 2024)	—	Expert-split; UCLBR inspiration
9	TIES-Merging (Yadav et al, NeurIPS 2023)	2306.01708	Paper B merging primitive
10	CL-LLM Survey (Shi et al, CSUR 2025)	2404.16789	Canonical taxonomy
11	Fisher bugs (Soen et al, 2025)	2502.11756	Critical for correct EWC + pruning

Papers NOT to cite as primary competitors (historical only)

- Progressive Neural Networks (Rusu et al 2016) — historical reference only
- EWC (Kirkpatrick et al PNAS 2017) — historical; not primary 2026 competitor
- DEN (Yoon et al ICLR 2018) — pre-transformer
- Model Soups (Wortsman et al ICML 2022) — parallel fine-tunes, misaligned

17. Training Data — Seq2Seq Completion

17.1 Why completion mode (not summarisation)

INCA is a **continual pre-training** system. The training signal must: (a) require no labels — the target is derived from the document itself, (b) have genuine temporal distribution shift — topics, vocabulary and facts change year over year, (c) be dense enough to saturate a block within ~20K documents per period.

In **completion mode** the document is split at a fixed fraction (default 50%):

```
input = "complete: " + article[:split_point] # encoder
target = article[split_point : split_point+200] # decoder (teacher-forced)
```

This is text-to-text compatible with FLAN-T5 and forces the model to build genuine sequence representations — not surface-pattern matching. Summarisation datasets (CNN/DM, XSum) have static (document, summary) pairs with **no temporal structure** and cannot test INCA's core capability.

17.2 Dataset ranking

Rank	Dataset	HuggingFace ID	Size	Date range	Temporal split	Phase
1 ★	CC-News	vblagoje/cc_news	708K articles (~2 GB)	2016–2019	Filter by date field → 6 half-year periods (H1/H2 per year)	Phase 1 — default
2	RedPajama-V2	togethercomputer/RedPajama-Data-V2	30T tokens (stream 20K per period)	2014–2023	84 CC snapshots → pick 6 snapshots, each = 1 period	Extension — temporal benchmark (requires streaming)
3	TiC-LM	apple/ml-tic-lm (GitHub)	2.9T tokens	May 2013–July 2024	114 monthly CC dumps; Apple's pipeline required	Paper-aligned benchmark

17.3 CC-News — period design

vblagoje/cc_news has a date field on every article. Split strategy: assign each article to one of 6 periods by (year, half). ~50–80K raw articles per period → subsample to 20K for Phase 1 experiments.

Period ID	Date range	Approx. articles	Typical content shift
P1 — 2017 H1	Jan–Jun 2017	~65K	Trump inauguration, early administration coverage
P2 — 2017 H2	Jul–Dec 2017	~70K	Tax reform, Harvey/Irma, Las Vegas shooting
P3 — 2018 H1	Jan–Jun 2018	~72K	Trade war starts, Cambridge Analytica, MeToo peak
P4 — 2018 H2	Jul–Dec 2018	~68K	Midterms, Saudi Khashoggi, crypto crash
P5 — 2019 H1	Jan–Jun 2019	~60K	Mueller report, Notre Dame, Boeing 737 MAX
P6 — 2019 H2	Jul–Dec 2019	~55K	Impeachment proceedings, HK protests, Brexit deadlock

17.4 Data loader (reference implementation)

```
from datasets import load_dataset
from datetime import datetime

ds = load_dataset("vblagoje/cc_news", split="train")

def assign_period(example):
d = datetime.fromisoformat(example["date"][:10])
```

```
key = f"{d.year}_H{'1' if d.month <= 6 else '2'}"
return {"period": key}

ds = ds.map(assign_period, num_proc=4)

PERIODS = ["2017_H1","2017_H2","2018_H1","2018_H2","2019_H1","2019_H2"]

def make_seq2seq(example, split_frac=0.50):
words = example["text"].split()
mid = max(30, int(len(words) * split_frac))
return {
"input_text": "complete: " + " ".join(words[:mid]),
"target_text": " ".join(words[mid : mid + 200]), # ~200 word target
}

period_datasets = {
p: ds.filter(lambda x: x["period"] == p)
.shuffle(seed=42).select(range(20_000))
.map(make_seq2seq, remove_columns=ds.column_names)
for p in PERIODS
}
```

17.5 RedPajama-V2 — streaming setup for E-ROUTE

```
from datasets import load_dataset
import itertools

# 6 CC snapshots ~6 months apart
SNAPSHOTS = ["2023-06","2022-49","2022-12","2021-43","2021-04","2020-24"]

def stream_period(snapshot, n=20_000):
ds = load_dataset(
"togethercomputer/RedPajama-Data-V2",
snapshots=[snapshot], languages=["en"],
name="default", streaming=True, split="train",
)
return list(itertools.islice(ds, n))

# snapshot → list[dict] — materialise one period at a time
period_data = {s: stream_period(s) for s in SNAPSHOTS}
```

17.6 Seq2seq tokenisation for FLAN-T5

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-base")

def tokenize(batch):
model_inputs = tokenizer(
batch["input_text"],
max_length=256, truncation=True, padding="max_length",
)
labels = tokenizer(
text_target=batch["target_text"],
max_length=256, truncation=True, padding="max_length",
)
# Replace pad token id in labels with -100 so loss ignores padding
labels["input_ids"] = [
```

```
[(l if l != tokenizer.pad_token_id else -100) for l in lbl]
for lbl in labels["input_ids"]
]
model_inputs["labels"] = labels["input_ids"]
return model_inputs

tokenized = {p: ds.map(tokenize, batched=True, batch_size=256)
for p, ds in period_datasets.items()}
```

Token budget per period: 20K articles × 512 tokens (input+target) = ~10M tokens. FLAN-T5-base trains at ~3K tokens/sec on A100 → ~55 min/period at batch 32. Full 6-period Phase 1 run: ~5.5 GPU-hours (excluding validation passes).

17.7 Why not summarisation or QA datasets

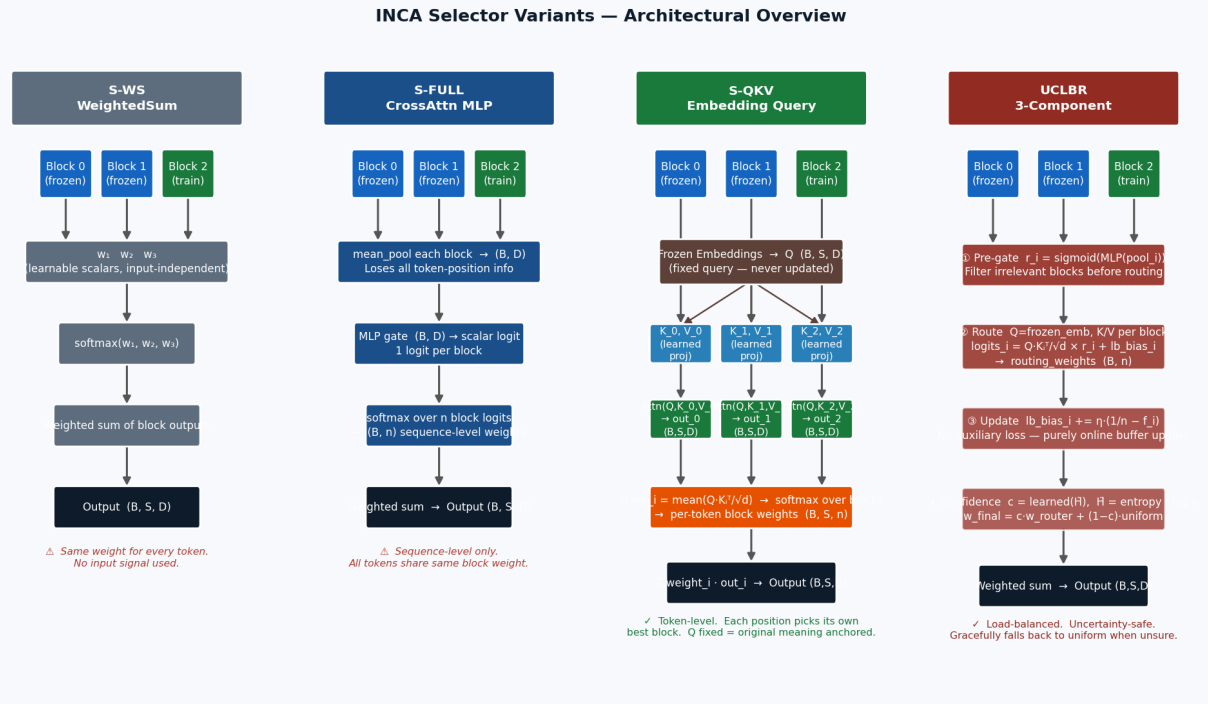
Dataset type	Temporal structure	Signal density	Verdict
CNN/DM, XSum	None — static pairs	High	No — wrong task; no drift
RealtimeQA	Weekly (news MC)	Low (30–35% F1 ceiling)	No — too sparse for pretraining
TemporalWiki	Year-half	Medium	Secondary eval only, not pretraining
CC-News (completion)	Half-year	High	YES — primary Phase 1 dataset
RedPajama-V2 (completion)	Snapshot-monthly	High	YES — E-ROUTE benchmark
TiC-LM (completion)	Monthly	Very high	YES — Paper A final benchmark

INCA — Selector Architecture

Deep dive: S-WS · S-FULL · S-QKV · UCLBR

INCA's selector aggregates the outputs of all blocks — frozen and trainable — into a single (B, S, D) tensor that the T5 decoder attends to. Four variants are implemented from Phase 1. All are ablated in experiment E-ROUTE. This section provides full architecture diagrams for each variant and explains the design decisions behind the recommended default (S-QKV).

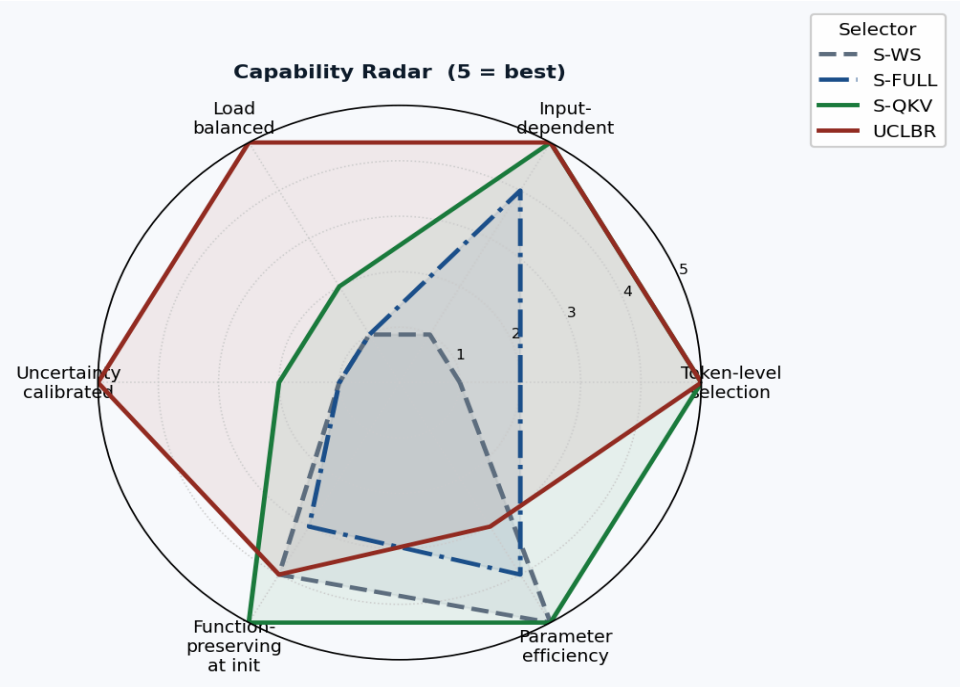
1. Overview — All Four Selectors



Left to right: S-WS (blind scalar weights) · S-FULL (MLP gate, sequence-level) · S-QKV (fixed-query, token-level, per-block separate) · UCLBR (full three-component router)

Selector	YAML key	Granularity	Blocks attended separately?	Recommended use
S-WS	weighted_sum	Sequence	N/A (no attention)	Control ablation only
S-FULL	cross_attention	Sequence	No (mean-pool then MLP)	Ablation baseline
S-QKV	embedding_query	Token-level	Yes ✓	Default for Paper A
UCLBR	uclbr	Token-level	Yes ✓	Extended experiments

2. Capability Radar



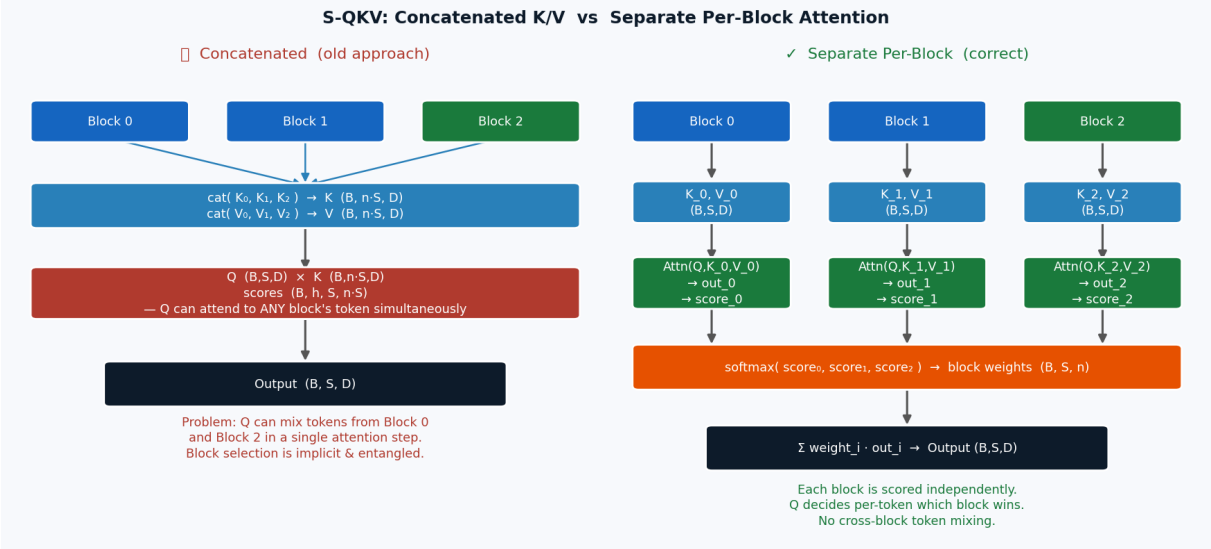
S-QKV and UCLBR both achieve maximum scores on token-level selection and input-dependence. UCLBR adds load-balancing and uncertainty calibration at the cost of implementation complexity and parameter count. S-WS and S-FULL are included as controls — they should be clearly outperformed by S-QKV if token-level routing is genuinely useful.

3. S-QKV — EmbeddingQuerySelector

Core design principle

Q is the frozen embedding — the original meaning of each token before any block has processed it. K and V come from each block's output. The selector asks: *"Given what this token originally meant, how well does each block's representation of it serve as an answer?"* The block with the highest Q·K■ alignment wins the most weight.

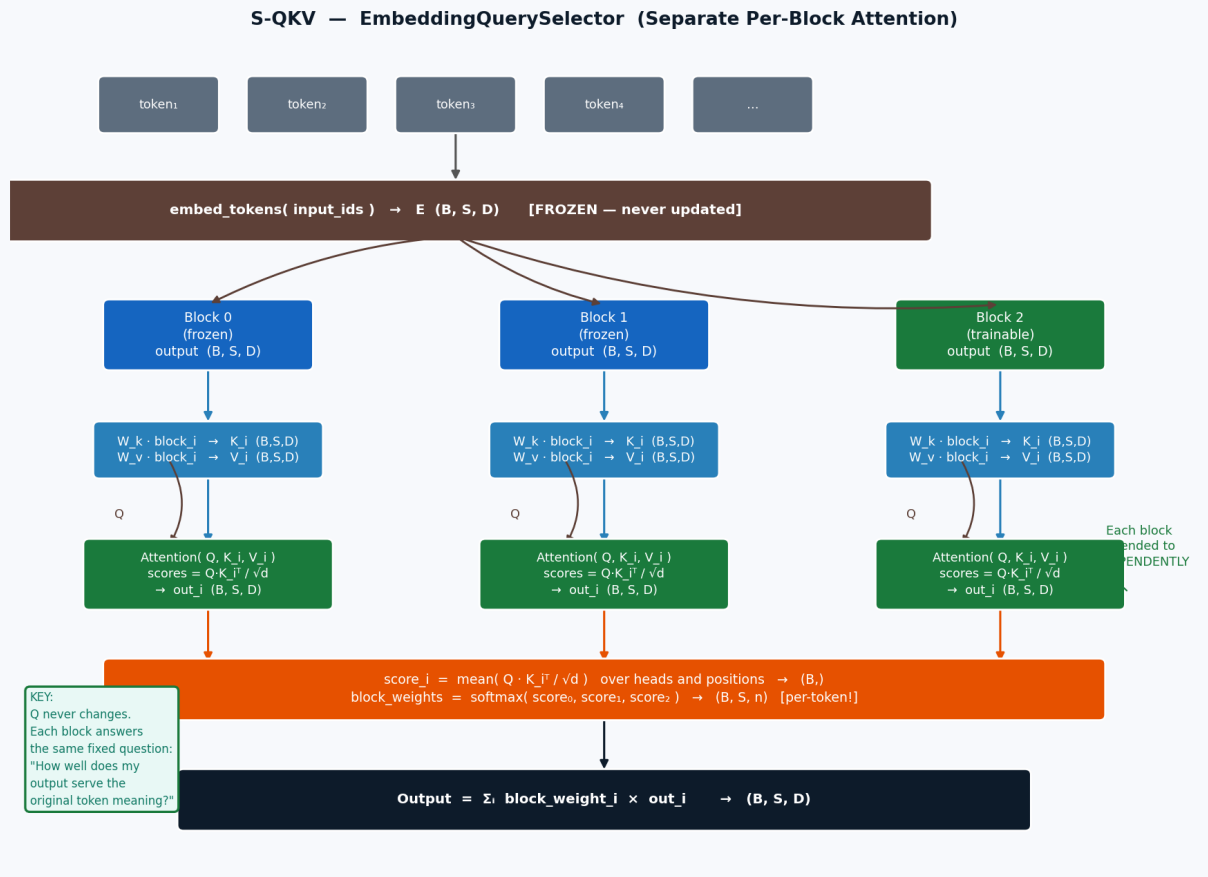
Key design choice: per-block separate attention (not concatenated)



The original (incorrect) implementation concatenated all block K/V tensors into one big matrix: K = cat(K■, K■, K■). This allowed Q to attend to tokens from Block 0 and Block 2 simultaneously in a single attention step, making block selection implicit and entangled. The correct design attends to each block independently:

- For each block i: compute Attention(Q, K_i, V_i) separately → out_i (B, S, D)
- Derive a block relevance score from the mean Q·K_i■ alignment
- Softmax over block scores → per-token block weights (B, S, n)
- Final output = Σ■ weight_i · out_i

Full data flow



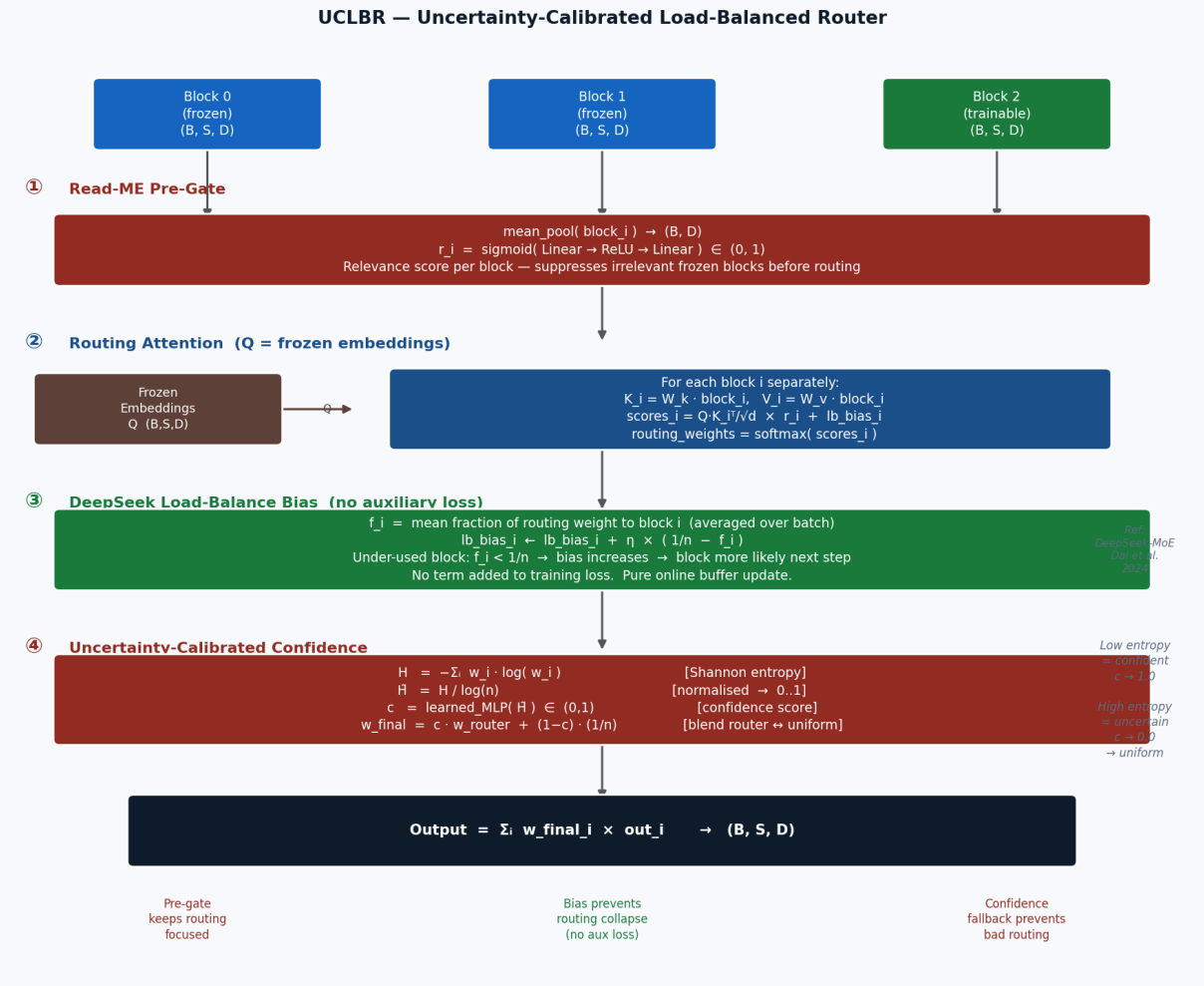
● **Q is never projected.** Raw frozen embeddings are used directly as queries. The question is always the same: what the token originally meant.

● **K and V are identity-initialised.** At grow time the selector is function-preserving — it outputs approximately the mean of all block outputs before any training.

● **Per-token block weights (B, S, n).** Token at position 3 can favour Block 0 while token at position 15 favours Block 2. This is the key advantage over S-FULL.

● **No gradient through Q.** Only W_k, W_v, W_out are learnable. The selector learns which blocks serve which kinds of tokens, not what the tokens are.

4. UCLBR — Three-Component Router



Component 1 — Read-ME Pre-gate

Before computing full attention, a two-layer MLP scores each block by mean-pooling its output and computing a sigmoid relevance score r_i ∈ (0,1). Routing logits are multiplied by r_i before softmax. Frozen blocks that have not yet specialised to the current task are suppressed, preventing their noise from diluting the routing signal of the relevant blocks.

Component 2 — DeepSeek Auxiliary-Loss-Free Load Balancing

Standard MoE load balancing appends an auxiliary loss to the training objective. This can destabilise training by fighting the primary language modelling objective. UCLBR uses the DeepSeek approach: a per-block bias buffer updated purely online after each forward pass — no gradient, no loss term. Under-used blocks accumulate positive bias; over-used blocks are penalised. Load balances naturally.

Component 3 — Uncertainty-Calibrated Confidence

High-entropy routing weights signal that the router is uncertain. When uncertain, committing to a bad routing decision is worse than spreading weight uniformly. The normalised entropy H■ = H/log(n) drives a learned confidence score c. The final weights blend the router's decision with a uniform fallback: the more uncertain, the more uniform the output. This prevents catastrophic routing errors during early training or on out-of-distribution

inputs.

5. E-ROUTE — Ablation Protocol

Run all four selectors on CC-News (6 periods, 3 seeds each). Pick the winner by BWT for the TiC-LM main run. The minimum bar: S-QKV must beat S-FULL by ≥0.5% BWT absolute, proving token-level routing is worth the complexity.

YAML key	Selector	Hypothesis	Pass condition
weighted_sum	S-WS	Lower bound — input routing adds no value over fixed weights	BWT < S-FULL
cross_attention	S-FULL	Sequence-level gating beats blind weights	BWT > S-WS
embedding_query	S-QKV	Token-level separate-block attention beats sequence-level gate	BWT > S-FULL by ≥0.5%
uclbr	UCLBR	Load balancing + uncertainty adds further gain over S-QKV	BWT ≥ S-QKV

Decision rule: if S-QKV ≥ UCLBR on BWT, use S-QKV as the Paper A default (simpler story, fewer moving parts). If UCLBR wins by ≥1%, use UCLBR as default and describe S-QKV as the ablated simplified version. Report both in Table 2.